

SDA All Lab Reports

COURSE CODE: SE-302L

COURSE TITLE:

Software Design & Architecture

LAB TITLE(s):

Report Serial No.	Lab Title
01	Familiarization with Microsoft Visio Studio
02	Domain Model
03	Use Case Diagrams - I
04	Use Case Diagrams - II
05	Modular Design of Hospital Management System
06	Class Diagram
07	Sequence Diagram
08	Activity Diagram
09	Deployment Diagrams
10	Package Diagram
11	Open-Ended Lab
12	(MVC) Implementation

Submitted To: Engr. Nighat Usman.

Submitted By: Muhammad Danial, Zameer Bukhari, Abdur Rehman, Mubashir Hassan, Umer Basit, Jalal Khan.

Registration – No.s: 24ABSWE0021, 24ABSWE0032, 24ABSWE0025, 24ABSWE0009, 24ABSWE0003, 24ABSWE0024.

Semester: 4th.

Dept. of Software Engineering, UET Peshawar, Abbottabad Campus.

Instructor Sign : _____



Labs Report # 01

Familiarization with Microsoft Visio Studio

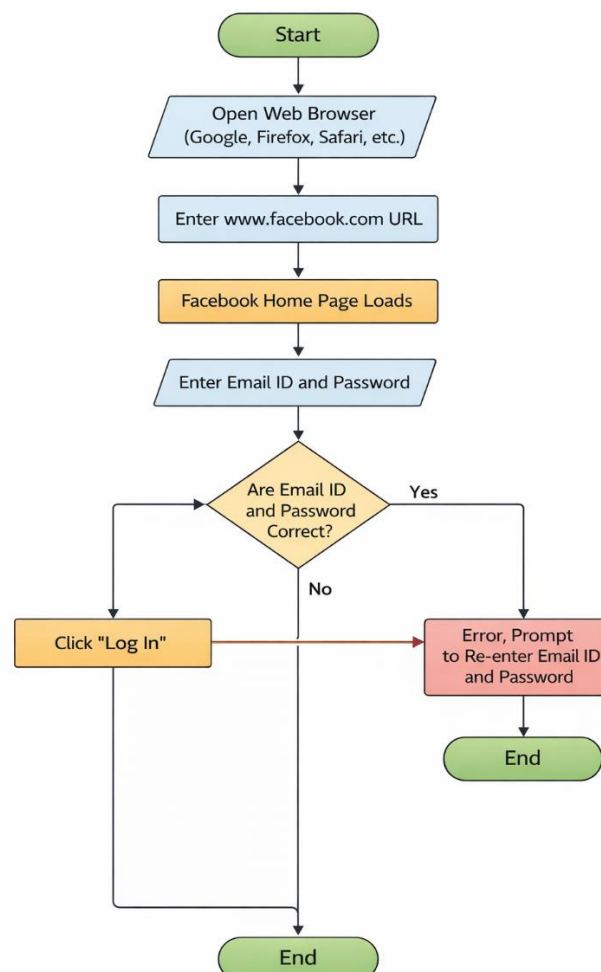
Lab Tasks

1. Familiarize yourself with menu on top bar (File, Edit etc.).
2. **Draw and submit a flowchart to log in to Facebook account**

The program works as follows:

- ✓ To log in to Facebook account, we first enter the Facebook URL www.facebook.com in our browser like Google, Firefox, Safari, Internet Explorer etc.
- ✓ This request is sent to the Facebook server and it responds by sending us the home page of Facebook.
- ✓ Next, we enter our registered Email ID and Password and click the Login button.
- ✓ Then our login credential is checked.
- ✓ If it is correct, we are shown our profile. On the other hand, if the login credential is wrong then an error occurs and we are prompted to re-enter our Email ID and Password.

Flowchart Design:





Lab Report # 02

DOMAIN MODEL

Lab Objective:

In this lab, students will learn how to identify real-world objects, their attributes, associations and how to model all this using Domain Model.

Case Study: BATS System

The Bank Accounts and Transactions System (BATS) is to be built for the Prime Bank Corporation. It must handle clients' bank accounts and the services on these accounts, i.e., deposit, withdraw, transfer, get balance, etc. The transactions are recorded, because at the end of each month, the system sends out account statements to all clients showing all transactions performed for their accounts during the last period. The system sends the statements to the printer from where a junior clerk posts them. The system is accessed by the bank's clients only indirectly, i.e., either via a teller, or an ATM, or the Internet. All transactions and queries are possible via a teller; all transactions and queries are possible except deposits via an ATM; and all except deposits and withdrawals via the Internet. Opening an account can be performed only via a teller and the Internet; however, if a client opens an account via the Internet they must identify themselves with a teller to have their account activated (this is government policy to avoid money laundering, e.g.). Closing an account can only be performed by a teller, and it requires a final statement to be sent out to the client. The Bank offers various account types, which fall into two categories: savings and checking. Savings accounts cannot be overdrawn. There can be a credit limit, subject to agreement by the bank, on checking accounts; a checking account cannot be overdrawn beyond this limit.

Lab Tasks:

Develop a partial Domain model for the given BATS system. Identify the concepts, attributes, and associations explicitly. This lab task must include:

1. List of possible conceptual classes
2. Attributes for each class
3. A partial domain model showing the conceptual classes and association between them.

1. List of Possible Conceptual Classes

The following conceptual classes were identified from the BATS system case study:

- Client.
- Account.
- Savings Account.
- Checking Account.
- Transaction.
- Statement.
- Teller.
- ATM.
- Internet Banking.
- Printer.
- Junior Clerk.



2. Attributes for each class

1) Client

- clientID
- clientName
- contactNumber
- address

2) Account

- accountNumber
- accountType
- balance
- status
- openingDate

3) Savings Account

- minimumBalance
- interestRate

4) Checking Account

- creditLimit
- overdraftAmount

5) Transaction

- transactionID
- transactionDate
- transactionType
- amount
- deposit
- withdrawal
- transfer
- balanceInquiry

6) Statement

- statementID
- statementDate
- ststatementPeriod
- transactionSummary

7) Teller

- tellerID
- tellerName
- branchID

8) ATM

- atmID
- location
- machineStatus



9) Internet Banking

- loginID
- password
- accountActivationStatus

10) Printer

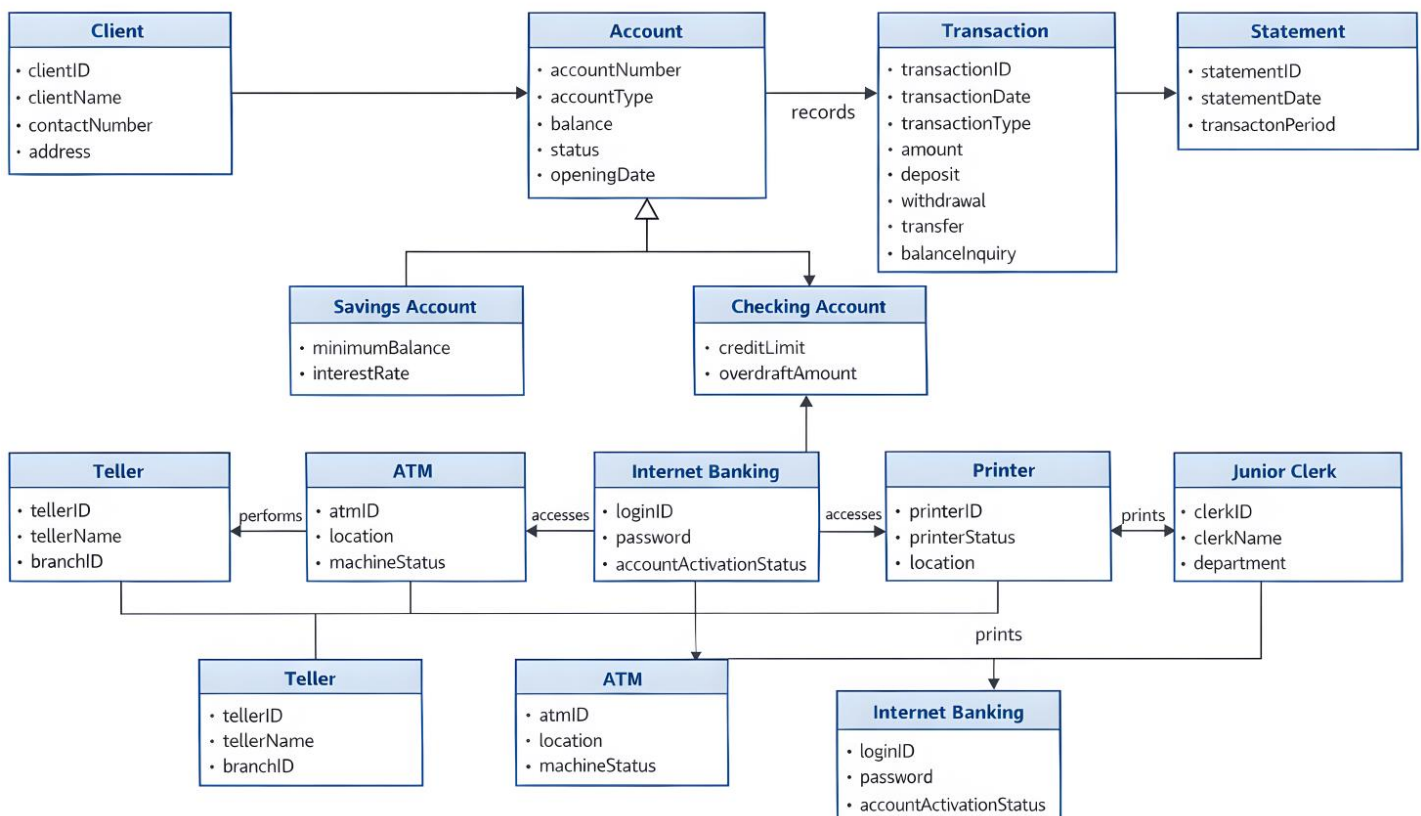
- printerID
- printerStatus
- location

11) Junior Clerk

- clerkID
- clerkName
- department

3. BATS system UML class diagram

BATS System – Domain Model





Lab Report # 03

USE CASE DIAGRAMS - I

Lab Objectives

- Analyze the requirements of the **Hurry's POS and Stock Control System**.
- Identify the **actors** and **use cases** involved in the system.
- Design a **UML Use Case Diagram** in **Microsoft Visio** to represent the system's functional behavior.

Case Study:

Hurry's require a new point of sale and stock control system for their many stores throughout the UK to replace their ageing mini based systems. A sales assistant will be able to process an order by entering product numbers and required quantities into the system. The system will display a description, price and available stock. In-stock products will normally be collected immediately by the customer from the store but may be selected for delivery to the customer's home address for which there will be a charge. If stock is not available, the sales assistant will be able to create a backorder for the product from a regional warehouse. The products will then either be delivered direct from the regional warehouse to the customer's home address or to the store for collection by the customer. The system will allow products to be paid for by cash or credit card. Credit card transactions will be validated via an online card transaction system. The system will produce a receipt. Order details for in-stock products will be printed in the warehouse including the bin reference, quantity, product number and description. These will be collected by the sales assistant and given to the customer. The sales assistant will be able to make refunds, provided a valid receipt is produced. The sales assistant will also be able to check stock and pricing without creating an order and progress orders that have been created for delivery. The store manager will be able at any time to print a summary report of sales in the store for a given period, including assignment of sales to sales assistants in order to calculate weekly sales bonuses. The stock manager will be able to monitor stock levels and weekly run-rates to set minimum stock levels and requisition products which fall below the minimum stock levels or for which demand is anticipated. When the stock arrives it will be booked in by the warehouse person. Stock that has been backordered for collection from the store is held in a separate area and the store manager advised of its arrival. The catalogue of available products will be maintained remotely by marketing from head office. Marketing will also be able to access sales information from each store system.

Lab Tasks

- Enlist all the possible Actors in the above scenario.
- Enlist all the possible Use Cases in the above scenario.
- Draw the detailed use-case diagram of the above requirement specifications.

Task 1: Actors (Actors involved in the Hurry's POS & Stock Control System)

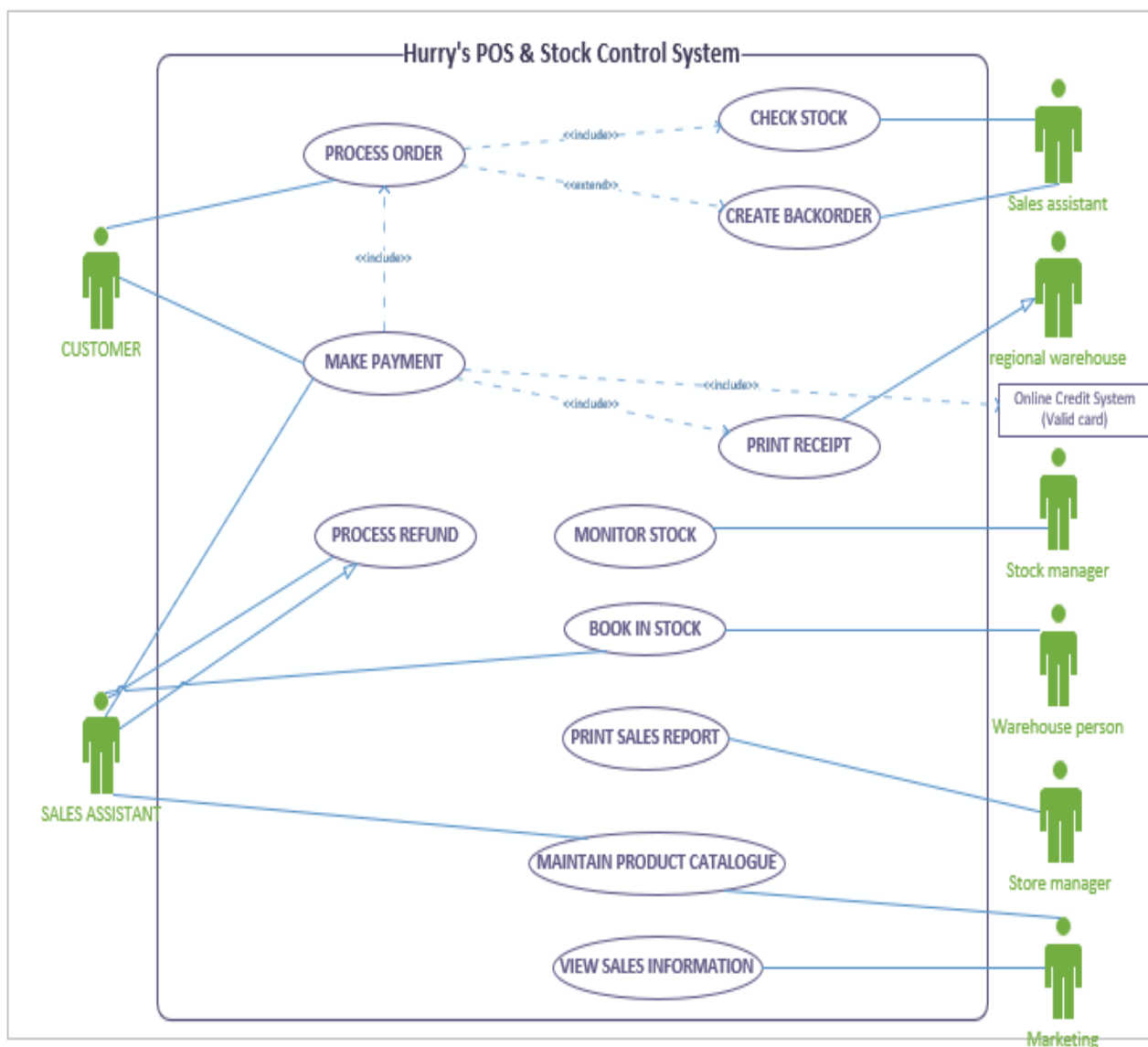
1. Customer.
2. Sales Assistant.
3. Regional Warehouse.
4. Online Credit Card System.
5. Stock Manager.
6. Warehouse Person.
7. Store Manager.
8. Marketing.



Task 2: Use Cases

1. Process Order.
2. Check Stock.
3. Create Backorder.
4. Make Payment.
5. Print Receipt.
6. Validate Credit Card.
7. Process Refund.
8. Monitor Stock.
9. Book In Stock.
10. Print Sales Report.
11. Maintain Product Catalogue.
12. View Sales Information.

Task 3: Use-case diagram





Lab Report # 04

USE CASE DIAGRAMS – II

Objective:

The objective of this lab is to learn how to prepare **Use Case Descriptions (Use Case Narratives)** and represent the functional requirements of an **ATM System** using a **Use Case Diagram**. The lab helps students understand the interaction between actors and the system by identifying use cases, defining normal, alternative, and exception flows, and documenting the system behavior based on the given ATM case study.

Case Study:

An Automated Teller Machine (ATM) allows customers to perform basic banking transactions. Before performing any transaction, the customer must authenticate by entering a valid account number and PIN. After successful authentication, the customer can view the account balance, withdraw cash, or transfer funds. The ATM communicates with the bank database to verify customer information and update account records.

Functional Requirements:

Requirement ID	Requirement Description
FR-01	Authenticate User
FR-02	View Account Balance
FR-03	Withdraw Cash
FR-04	Transfer Funds
FR-05	Validate Account Number
FR-06	Verify PIN
FR-07	Retrieve Balance from Database
FR-08	Update Account Balance

Actors:

Actor	Description
Customer	Performs ATM transactions such as balance inquiry, cash withdrawal and fund transfer.
Bank Database	Verifies account number, PIN, stores balance information and updates customer records.

ATM Session:

The ATM session begins when the customer approaches the ATM. The customer enters the account number followed by the PIN. The ATM verifies the entered information with the bank



database. If the authentication is successful, the main menu is displayed. The customer may then perform one or more transactions such as viewing the account balance, withdrawing cash or transferring funds. After completing the transaction, the ATM returns to the main menu. When the customer selects Exit, a thank-you message is displayed and the ATM returns to the welcome screen for the next customer.

Use Case Narrative:

A Use Case Narrative provides the detailed description of each use case. It specifies the interaction between the actor and the system, including the normal flow, alternative flow and exception flow.

Use Case Description – Authenticate User:

Field	Description
Use Case ID	UC-01
Use Case Name	Authenticate User
Actors	Customer, Bank Database
Created By	Student
Last Updated By	Student
Date Created	_____
Date Last Updated	_____
Description	Allows the customer to access the ATM by entering a valid account number and PIN.
Preconditions	ATM is ON and connected to the Bank Database.
Postconditions	Customer is authenticated and the Main Menu is displayed.

Normal Flow:

Step	Action
1	ATM displays Welcome Screen.
2	Customer enters Account Number.
3	ATM prompts for PIN.
4	Customer enters PIN.
5	ATM verifies Account Number and PIN with the Bank Database.
6	If valid, Main Menu is displayed.

Alternative Flow:

Step	Action
1	Invalid Account Number entered.
2	ATM displays an error message.
3	Customer enters the Account Number again.



Exception Flow:

Step	Action
1	Incorrect PIN entered.
2	ATM displays "Invalid PIN".
3	Authentication process restarts.

Use Case Description – View Balance:

Field	Description
Use Case ID	UC-02
Use Case Name	View Balance
Actors	Customer, Bank Database
Description	Displays the available account balance.
Preconditions	Customer is authenticated.
Postconditions	Current balance is displayed on the screen.

Normal Flow:

Step	Action
1	Customer selects View Balance.
2	ATM requests balance from Bank Database.
3	Database returns balance.
4	ATM displays account balance.
5	ATM returns to Main Menu.

Alternative Flow:

No alternative flow.

Exception Flow:

Database connection fails. ATM displays an appropriate error message.

Use Case Description – Withdraw Cash

Field	Description
Use Case ID	UC-03
Use Case Name	Withdraw Cash
Actors	Customer, Bank Database
Description	Allows the customer to withdraw cash from the account.
Preconditions	Customer is authenticated and ATM contains cash.
Postconditions	Cash is dispensed and account balance is updated.



Normal Flow:

Step	Action
1	Customer selects Withdraw Cash.
2	ATM displays withdrawal menu.
3	Customer enters withdrawal amount.
4	ATM requests PIN for verification.
5	ATM checks account balance.
6	ATM checks cash availability.
7	ATM deducts amount from account.
8	ATM dispenses cash.
9	ATM reminds customer to collect cash.
10	Main Menu is displayed.

Alternative Flow:

Condition	System Response
Withdrawal amount exceeds balance	Display "Insufficient Balance". Ask customer to enter a smaller amount or exit.
ATM has insufficient cash	Display "ATM has insufficient cash". Return to Withdrawal Menu.

Exception Flow:

If the database connection fails, the transaction is cancelled and an error message is displayed.

Use Case Description – Transfer Funds:

Field	Description
Use Case ID	UC-04
Use Case Name	Transfer Funds
Actors	Customer, Bank Database
Description	Transfers money from one account to another.
Preconditions	Customer is authenticated and destination account exists.
Postconditions	Amount is transferred successfully.

Normal Flow:

Step	Action
1	Customer selects Transfer Funds.
2	ATM requests destination account number.
3	Customer enters account number.
4	Customer enters transfer amount.



Step	Action
5	ATM verifies available balance.
6	ATM transfers amount.
7	Database updates both account balances.
8	Success message is displayed.
9	Main Menu is displayed.

Alternative Flow:

Condition	System Response
Insufficient Balance	Display error message and cancel transfer.

Exception Flow:

If the destination account is invalid, the transfer is cancelled and an error message is displayed.

Lab Task:

Draw the detailed use-case diagram and description keeping in mind the requirement specifications described above.

Use Case Diagram for ATM System:(MS Visio)



Conclusion:

In this experiment, a detailed **Use Case Diagram** and **Use Case Narratives** were prepared for the ATM System. The interactions between the Customer and the Bank Database were identified, and the complete flow of authentication, balance inquiry, cash withdrawal, and fund transfer was documented. This experiment helped in understanding how UML Use Case Diagrams and Narratives are used to capture and document software requirements before system development.



Lab Report # 05

MODULAR DESIGN OF HOSPITAL MANAGEMENT SYSTEM

OBJECTIVE

The objective of this lab is to understand the concept of modular design and apply it to develop a modular structure for a Hospital Management System. This experiment focuses on dividing a complex software system into smaller, independent, and reusable modules to reduce complexity and improve maintainability.

INTRODUCTION

Modular design is a software and system engineering approach in which a large system is divided into smaller independent components called modules. Each module performs a specific functionality and can be developed, tested, modified, and reused independently.

A modular system improves software quality by reducing complexity and making future changes easier. It also allows multiple developers to work on different modules simultaneously.

The main characteristics of modular design include:

- Functional division of a system into separate and reusable modules.
- Well-defined interfaces between different modules.
- Easy modification and maintenance of system components.
- Reduction of system complexity through independent development.

Concepts of Modular Design

1. Functional Independence

Functional independence means that each module should perform a specific task with minimum dependency on other modules. A good module should have a clear purpose and should communicate with other modules through simple interfaces.

Functional independence is measured through two important concepts:

Cohesion

Cohesion describes the relationship between the functions inside a single module. A module with high cohesion performs one specific task effectively.

Example:

A Patient Management Module should only handle patient-related operations such as adding, updating, and searching patient records.

Coupling

Coupling describes the dependency between different modules. Lower coupling is preferred



because it makes the system easier to modify and maintain.

Example:

The Billing Module should communicate with the Patient Module only when patient information is required instead of depending completely on it.

Modular Design of Hospital Management System

A Hospital Management System is a large software system that manages hospital activities including patients, doctors, appointments, billing, and medical records.

The system can be divided into the following modules:

1. Patient Management Module

Purpose:

This module manages all patient-related information.

Functions:

- Register new patients
- Store patient information
- Update patient records
- Search patient details
- Maintain medical history

2. Doctor Management Module

Purpose:

This module manages doctor information and availability.

Functions:

- Add doctor details
- Update doctor profiles
- Manage doctor schedules
- View assigned patients

3. Appointment Management Module

Purpose:

This module handles patient appointments with doctors.

**Functions:**

- Book appointments
- Cancel appointments
- Check available time slots
- Maintain appointment records

4. Medical Records Module

Purpose:

This module stores and manages patient medical records.

Functions:

- Store diagnosis information
- Maintain treatment history
- Manage prescriptions
- View previous medical reports

5. Billing Management Module

Purpose:

This module manages hospital payments and financial records.

Functions:

- Generate bills
- Calculate treatment charges
- Manage payments
- Maintain billing history

6. Pharmacy Management Module

Purpose:

This module manages medicines and pharmacy operations.

Functions:

- Maintain medicine inventory
- Issue medicines
- Update stock information
- Manage medicine records



7. Staff Management Module

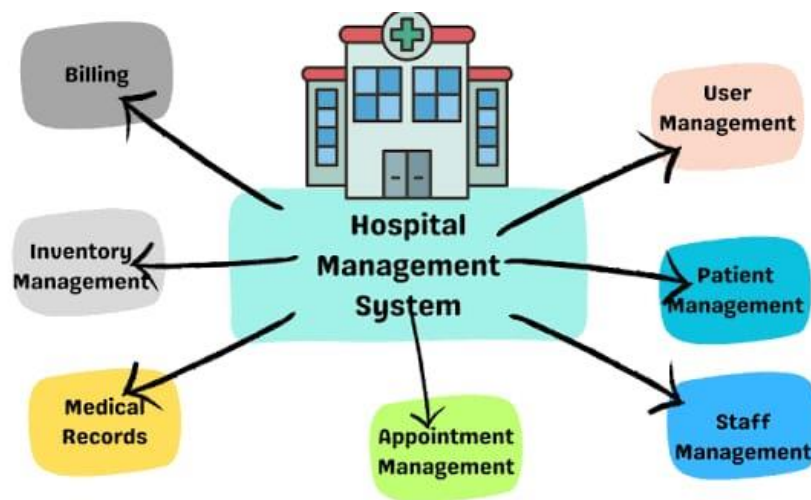
Purpose:

This module manages hospital employees.

Functions:

- Add staff information
- Manage employee records
- Maintain duty schedules

Hospital Management System Modular Architecture

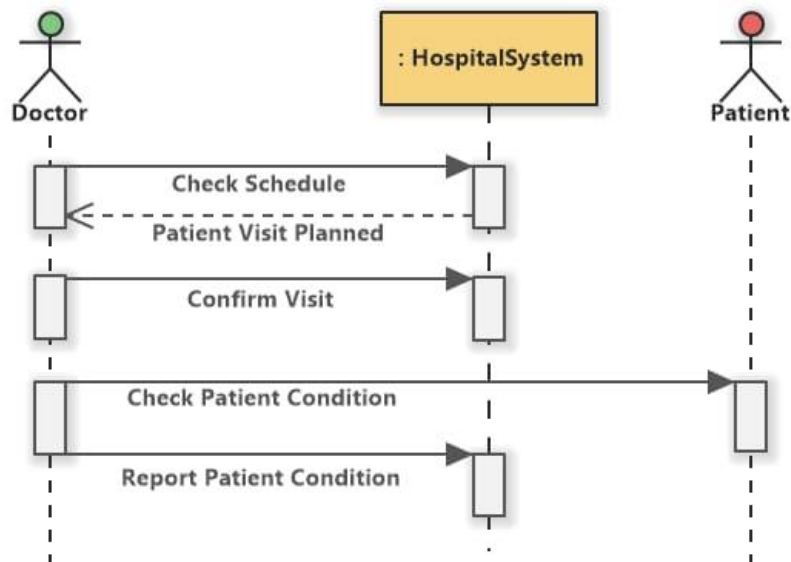


Module Interaction

The modules communicate with each other through defined interfaces.

Examples:

- Patient Module communicates with Appointment Module for booking appointments.
- Doctor Module communicates with Appointment Module to manage doctor schedules.
- Billing Module receives information from Patient and Medical Record Modules.
- Pharmacy Module provides medicine information related to patient treatment.



Advantages of Modular Design

The modular design approach provides several benefits:

1. Reduced Complexity

A large system is divided into smaller manageable modules, making development easier.

2. Easy Maintenance

Changes can be made in one module without affecting the complete system.

3. Reusability

Modules can be reused in other systems.

4. Parallel Development

Different developers can work on different modules at the same time.

5. Better Testing

Each module can be tested independently before integration.

6. Improved Reliability

Independent modules reduce the chances of system failure.



Conclusion

In this experiment, we studied the concept of modular design and applied it to a Hospital Management System. The system was divided into different independent modules such as Patient Management, Doctor Management, Appointment Management, Billing, Pharmacy, and Medical Records.

Modular design reduces software complexity, improves maintainability, and allows easier modification and development. A well-designed modular system provides better performance, flexibility, and scalability.

Lab Report # 06

CLASS DIAGRAM

Case Study:

Create a class diagram for a graphical document editor. Consider that document editor supports grouping. Document editor comprised of documents. Document has doc id, name, date of creation and date of update. User can create new document, save that document and print document. Assume that a document consists of several sheets. Sheet can be copied, edited and deleted. Each sheet contains drawing objects, including text, geometrical objects and groups. User can also create, group and format drawing objects. A group is simply a set of drawing objects which can be created or deleted. Text on the sheets has various font style and sizes that can be edited as per requirements. User can create, rotate and translate geometrical objects. A geometrical object includes circle, ellipse, rectangles, lines and squares, having their own areas.

Task:

Create a class diagram according to the case study. Also mention classes and relationships between each class

Table 1: Classes Description

Class Name	Type	Main Attributes	Main Methods/Responsibilities
User	Concrete	userID, name	createDocument(), saveDocument(), printDocument(), createDrawingObject(), groupObjects()
Document	Concrete	docID, name, creationDate, updateDate	addSheet(), removeSheet(), save(), print()
Sheet	Concrete	sheetID, sheetName	copy(), edit(), delete(), addObject(), removeObject()
DrawingObject	Abstract	objectID, color, position	draw(), delete()
Text	Concrete	content, fontStyle,	editText()



Class Name	Type	Main Attributes	Main Methods/Responsibilities
GeometricalObject	Abstract	area fontSize	create(), rotate(), translate(), calculateArea()
Circle	Concrete	radius	calculateArea()
Rectangle	Concrete	length, width	calculateArea()
Square	Concrete	side	calculateArea()
Ellipse	Concrete	majorAxis, minorAxis	calculateArea()
Line	Concrete	length	create(), calculateArea()
Group	Concrete	groupID	addObject(), removeObject(), createGroup(), deleteGroup()

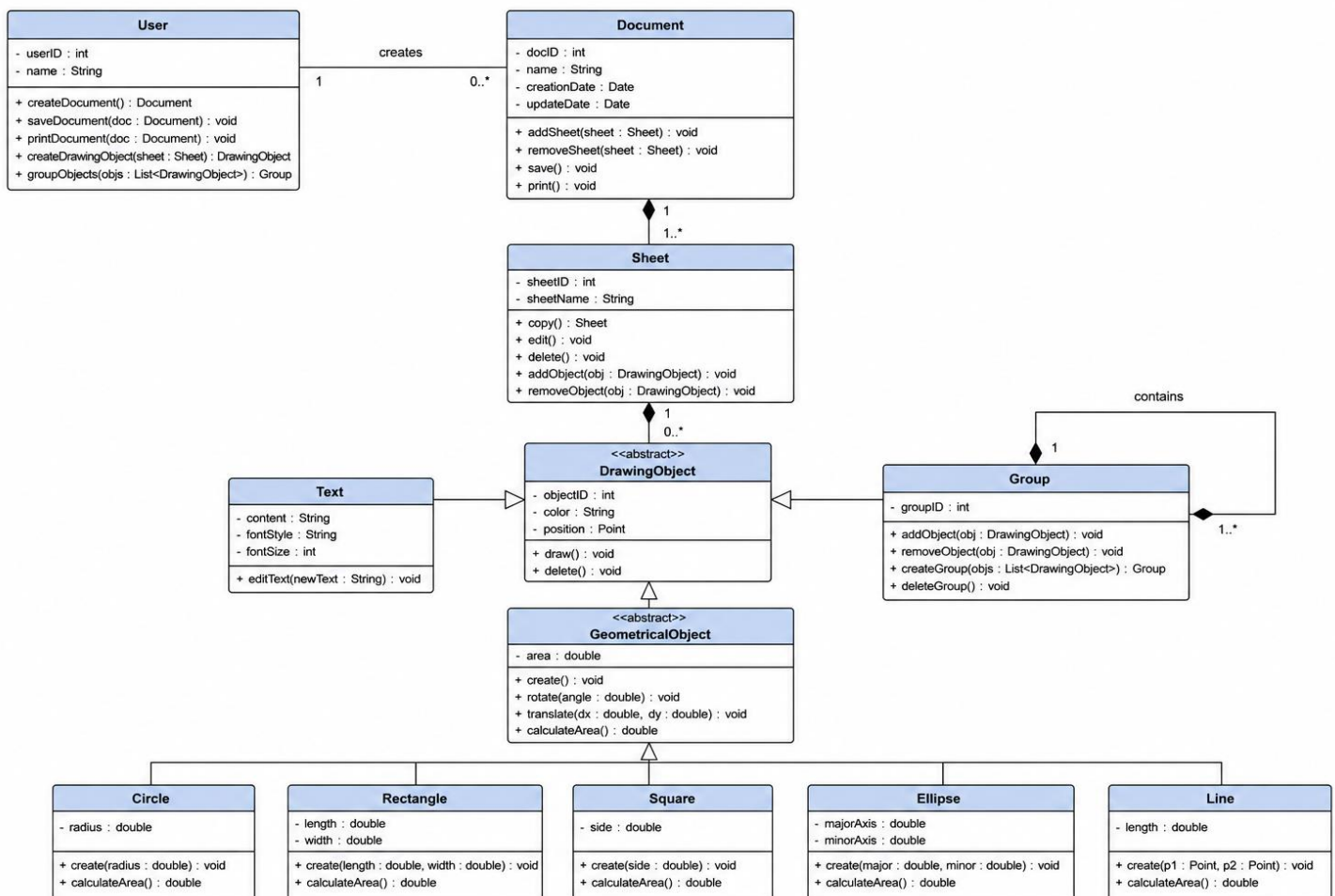
Table 2: Relationships Between Classes

Source Class	Destination Class	Relationship Type	Multiplicity	Description
User	Document	Association	1 → 0..*	A user can create and manage multiple documents.
Document	Sheet	Composition	1 → 1..*	A document consists of one or more sheets.
Sheet	DrawingObject	Composition	1 → 0..*	A sheet contains multiple drawing objects.
DrawingObject	Text	Inheritance	—	Text is a type of drawing object.
DrawingObject	GeometricalObject	Inheritance	—	GeometricalObject extends DrawingObject.
DrawingObject	Group	Inheritance	—	Group is also treated as a drawing object.
GeometricalObject	Circle	Inheritance	—	Circle inherits from GeometricalObject.
GeometricalObject	Rectangle	Inheritance	—	Rectangle inherits from GeometricalObject.
GeometricalObject	Square	Inheritance	—	Square inherits



Source Class	Destination Class	Relationship Type	Multiplicity	Description
t				from GeometricalObject .
GeometricalObject	Ellipse	Inheritance	—	Ellipse inherits from GeometricalObject .
GeometricalObject	Line	Inheritance	—	Line inherits from GeometricalObject .
Group	DrawingObject	Composition	1 → 1..*	A group contains one or more drawing objects.

Class Diagram:





Lab Report # 07

SEQUENCE DIAGRAM

1. Objective

The objective of this lab is to practice the construction of UML sequence diagrams, and to apply this knowledge by modeling the interaction behavior of a real-world case study, the ATM Cash Withdrawal system, using Microsoft Visio.

2. Sequence Diagrams

A UML sequence diagram models the flow of logic within a system in a visual manner, allowing both documentation and validation of the underlying logic. It describes an interaction among a set of objects participating in a scenario, arranged in chronological order, showing the participating objects as “lifelines” and the messages exchanged between them over time.

Sequence Diagram Notations

- Lifeline: Represents an individual participant in the interaction; drawn as a vertical dashed line.
- Actor: A role played by an entity that interacts with the system, typically a human user or external system.
- Activation Bar: A thin rectangle on a lifeline representing the period during which an element performs an operation; its top and bottom align with the start and end times of that operation.
- Call Message: Represents an invocation of an operation on the target lifeline.
- Return Message: Represents information passed back to the caller in response to an earlier call.
- Self-Message: Represents an operation that a lifeline invokes on itself.

3. Tool Used

Microsoft Visio was used to construct the sequence diagram for this lab. The diagram was created using the following steps:

1. Start MS Visio and select the “Software and Databases” category.
2. Select “UML Model Diagrams”.
3. Select “Sequence Diagrams” from the template options.
4. Add actor and lifeline shapes, using an actor notation from the use-case section combined with the “Life Line” object where required.
5. Add activation bars and connect the lifelines using call, return, and self-message arrows to represent the interaction.

4.Task

Case Study: ATM Cash Withdrawal



4.1 Brief Description

This use case describes how a Bank Customer uses an ATM to withdraw cash from their bank account.

4.2 Actors

- Bank Customer
- Bank

4.3 Pre-conditions

- There is an active network connection to the Bank.
- The ATM has cash available.

4.4 Basic Flow of Events

- The use case begins when the Bank Customer inserts their bank card.
- The Validate User use case is performed.
- The ATM displays the available transaction options; the Bank Customer selects “Withdraw Cash”.
- The ATM prompts for an account, and the Bank Customer selects one.
- The ATM prompts for an amount, and the Bank Customer enters it.
- The card ID, PIN, amount, and account are sent to the Bank as a transaction; the Bank replies with a go / no-go response.
- The money is dispensed.
- The bank card is returned.
- A receipt is printed, and the use case ends successfully.

4.5 Alternative Flows (Summary)

- Invalid User – ends with a failure condition if the Validate User use case does not complete successfully.
- Wrong Account – an “Invalid Account” message is shown and the flow resumes at account selection.
- Wrong Amount – the customer is asked to re-enter an amount that matches the available bill denominations.
- Amount Exceeds Withdrawal / Daily Limit – a warning is shown and the customer re-enters the amount.
- Insufficient Cash – a warning is shown if the ATM cannot fulfil the requested amount.
- No Response from Bank – the ATM retries up to three times before returning the card and displaying “Network unavailable.”
- Money Not Removed – the ATM issues a warning, then retracts the cash and logs the failure if it is not collected.
- Quit – the transaction is cancelled, a cancellation receipt is printed, and the card is returned.

4.6 Special Requirements

- The ATM dispenses cash only in multiples of \$20.
- The maximum individual withdrawal amount is \$500.



- The ATM keeps a log, including date and time, of all complete and incomplete transactions with the Bank.

5. Sequence Diagram

The sequence diagram below models the basic flow of the ATM Cash Withdrawal case study, showing the interaction between the Bank Customer, the ATM, and the Bank system:

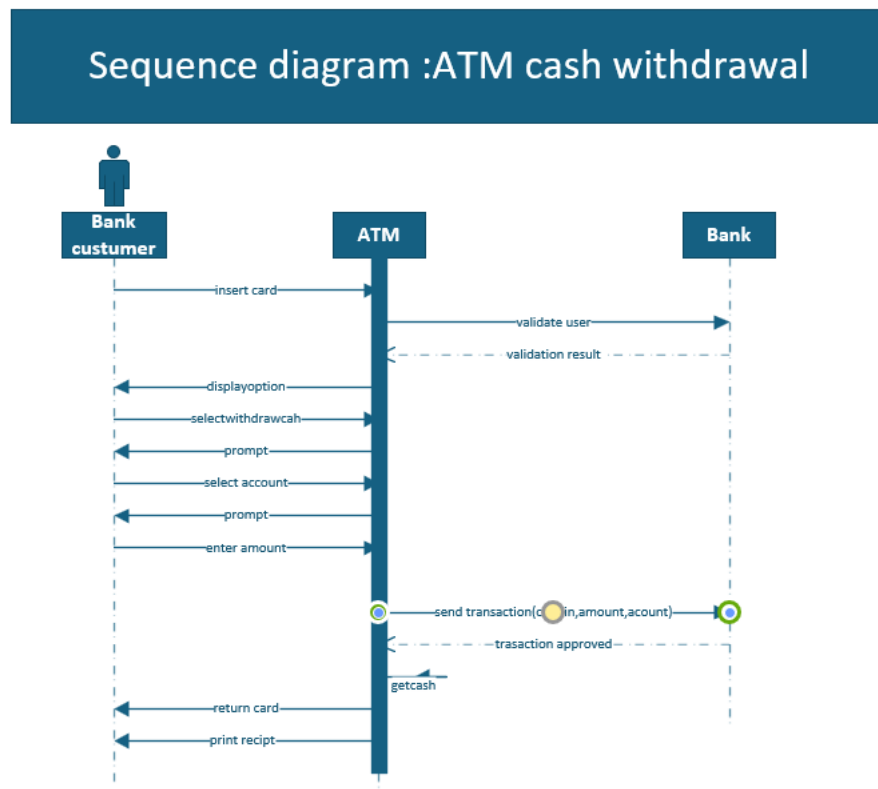


Figure 1 Sequence diagram

6. Diagram Explanation

The Bank Customer initiates the interaction by inserting their card into the ATM. The ATM forwards the card and PIN details to the Bank for validation, and the Bank returns the validation result. Once validated, the ATM displays the available options and guides the customer through selecting an account and entering a withdrawal amount, each of which triggers a call message and, where applicable, a return message back to the actor. The ATM then sends the complete transaction (card ID, PIN, amount, and account) to the Bank, which returns an approval message. On approval, the ATM performs a self-message to dispense the cash, before finally returning the card and printing the receipt for the customer. The activation bar on the ATM lifeline reflects that it remains the active controller throughout the transaction, coordinating messages between the customer and the bank.



7. Conclusion

Through this lab, a clear understanding was developed of how sequence diagrams capture the chronological interaction between objects and actors in a system. Modeling the ATM Cash Withdrawal case study demonstrated how call messages, return messages, and self-messages work together to represent both the primary flow and the coordination between an actor, a controller object (the ATM), and an external system (the Bank).

Lab Report # 08

ACTIVITY DIAGRAM

How to Create Activity Diagram

1. Go to File □ New □ Software & Database □ UML Model Diagram.
2. Select **UML Activity** stencil from Shapes Menu.
3. Drag an **Action State** or **State** shape onto the drawing page for each action or activity state you want to represent. Use the **Initial State** and **Final State** shapes to represent initial and final pseudo states.
4. Connect **Control Flow** shapes to **State** shapes to indicate the change from one state to another.

Indicate the flow of control in an activity diagram:

- a. In an activity diagram, drag a **Control Flow** shape onto the drawing page.
 - b. Glue the **Control Flow** shape endpoint (without the arrowhead) to a connection point  on the source **Action State** or **State** shape.
 - c. Glue the **Control Flow** shape endpoint (with an arrowhead) to a connection point on the destination **Action State** or **State** shape.
 - d. Double-click the **Control Flow** shape to add a transition string.
5. Use the complex transition shapes, **Transition (Fork)** or **Transition (Join)**, to represent the forking of one action state into multiple parallel states, or the synchronization of multiple action states into one state.
 6. Double-click any shape to open its **UML Properties** dialog box where you can add a name, transition string etc.
 7. Save the diagram.

ACTIVITY DIAGRAM NOTATIONS

Initial Node: The starting state before an activity takes place is depicted using the **initial state**. The Initial State from the UML Activity Diagram marks the entry point and the initial Activity State.

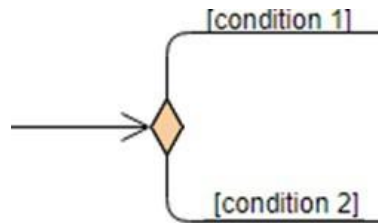




Final Node: In the UML, a final action state is shown using a circle surrounding a small solid filled circle (a bull's eye).



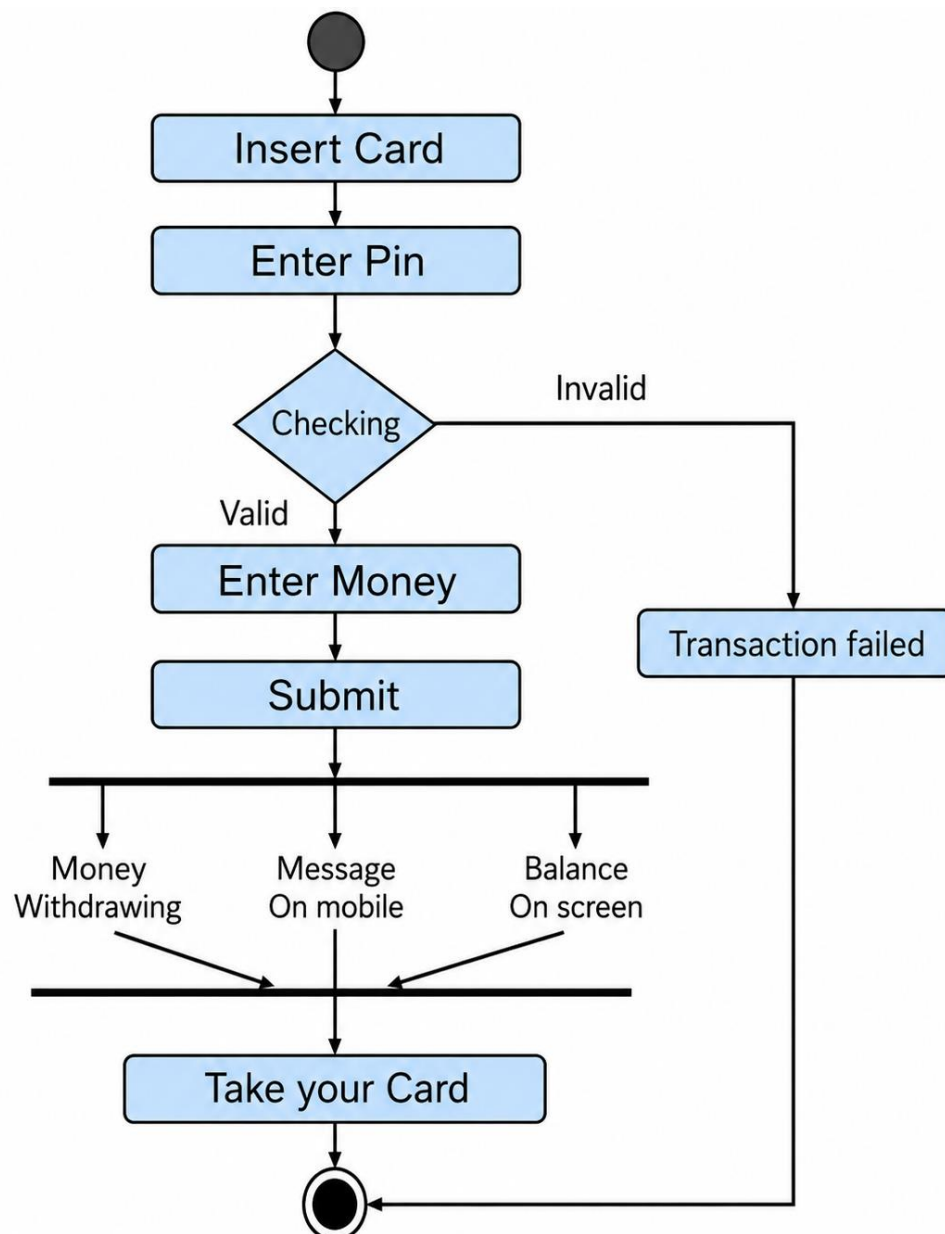
Decision Node: A decision node accepts tokens on an incoming edge and presents them to multiple outgoing edges.



Task

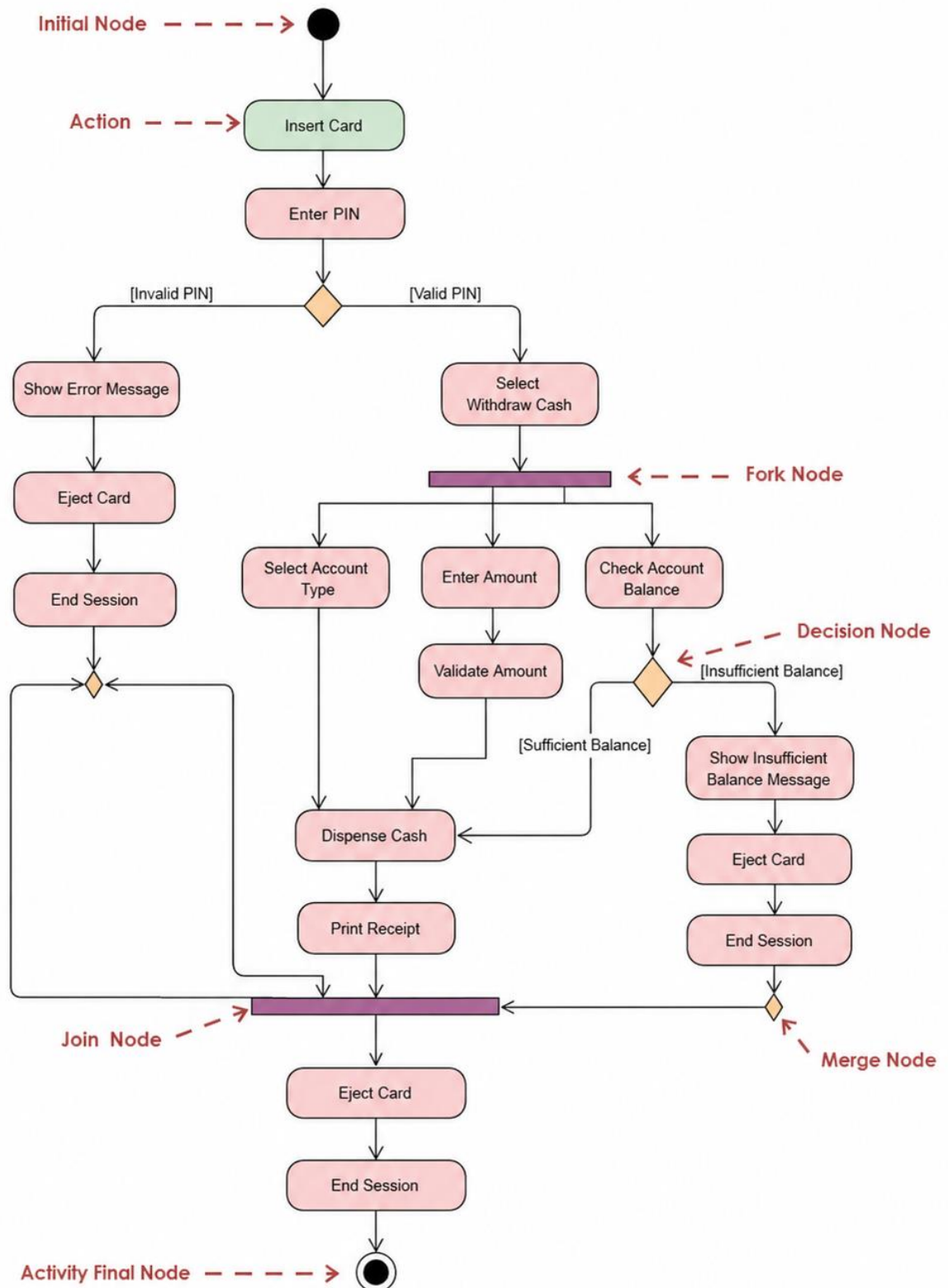
Create an activity diagram of the ATM Cash Withdrawal system explained in Experiment 6.

ATM Conceptual Activity Diagram





ATM Detailed Activity Diagram



Summary

Activity Diagrams are important for modeling system functions. They emphasize the flow of objects and synchronization of the flow in support of parallel processing. The activity diagram of the given case study showed important flows of the system.



Lab Report # 09

DEPLOYMENT DIAGRAMS

Objectives:

In today's lab, students will learn how to draw deployment diagram. First an introduction will be given; next students will practice an exercise related to deployment diagram.

Deployment Diagram:

A Deployment Diagram is a UML (Unified Modeling Language) diagram that represents the physical deployment of a software system. It illustrates the hardware devices (nodes), software components, and the communication links between them. Deployment diagrams help visualize how an application is installed and executed in a real-world environment, making them useful for understanding system architecture, network configuration, and the distribution of software across different devices or servers.

Deployment Diagram Notations:

Node: A node is a run-time physical object that represents a computational resource. It is denoted by a 3D box with the node-name written inside of it. Nodes help to convey the hardware which is used to deploy the software. You may model the component instances that run or live on a node by drawing them within the node. Node is a computational resource upon which artifacts are deployed for execution. A node is a physical thing that can execute one or more artifacts.

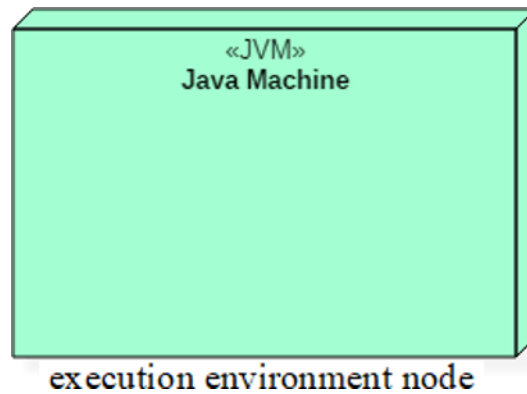
Generally, a node has two stereotypes as follows:

<<Device>>

It is a node that represents a physical machine capable of performing computations. A device can be a router or a server PC. It is represented using a node with stereotype <<device>>. In the UML model. Following is a representation of a device in UML:

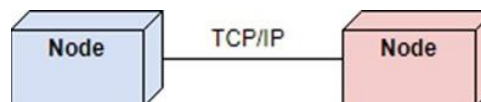


It is a node that represents an environment in which software is going to execute. For example, Java applications are executed in java virtual machine (JVM). JVM is considered as an execution environment for Java applications. We can nest an execution environment into a device node. Following is a representation of an execution environment in UML:

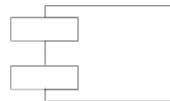


You may model which nodes communicate with one another using the Connection relationship line.

Connection: A connection depicts the communication path used by the hardware to communicate usually indicates the method i.e. TCP/IP.



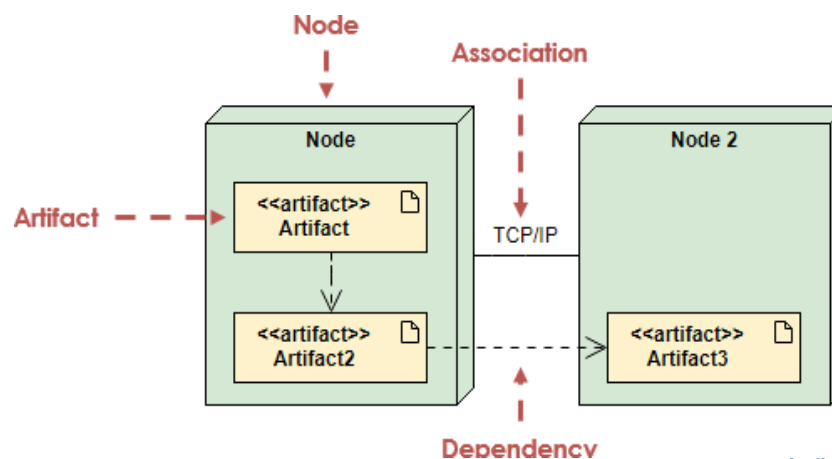
Component: A component is a grouping of classes that work together closely.



Artifact: Artifacts represent concrete elements in the physical world that are the result of a development process. An artifact represents the specification of a concrete real-world entity related to software development. You can use the artifact to describe an executable file. Artifacts are deployed on the nodes. The most common artifacts are as follows,

1. Source files.
2. Executable files.
3. Database tables/schemas.
4. Configuration files.
5. Libraries.
6. Scripts.
7. DLL files.
8. User manuals or documentation.
9. Output files.

Example:





Case Study:

Paradise introduces an online registration and reservation facilities to its customers. For that purpose, it wants to make a system for hotel business names as **Hotel Reservation System**. A Hotel Reservation multiple view models is to be created for a Software Product Line, which can be tailored to the needs of an individual hotel chain or hotel.

The system manages information about rooms, reservations, customers, and customer billing. This system allows for the hotel guest to search available rooms, make a reservation (after registration). Up to 2 reservations are possible for each guest. He may be able to confirm reservation, cancel reservation, modify reservation and pay through credit card or cash. When making a reservation through a reservation clerk, a customer gives personal details, states the room type, number of occupants, and dates of arrival and departure.

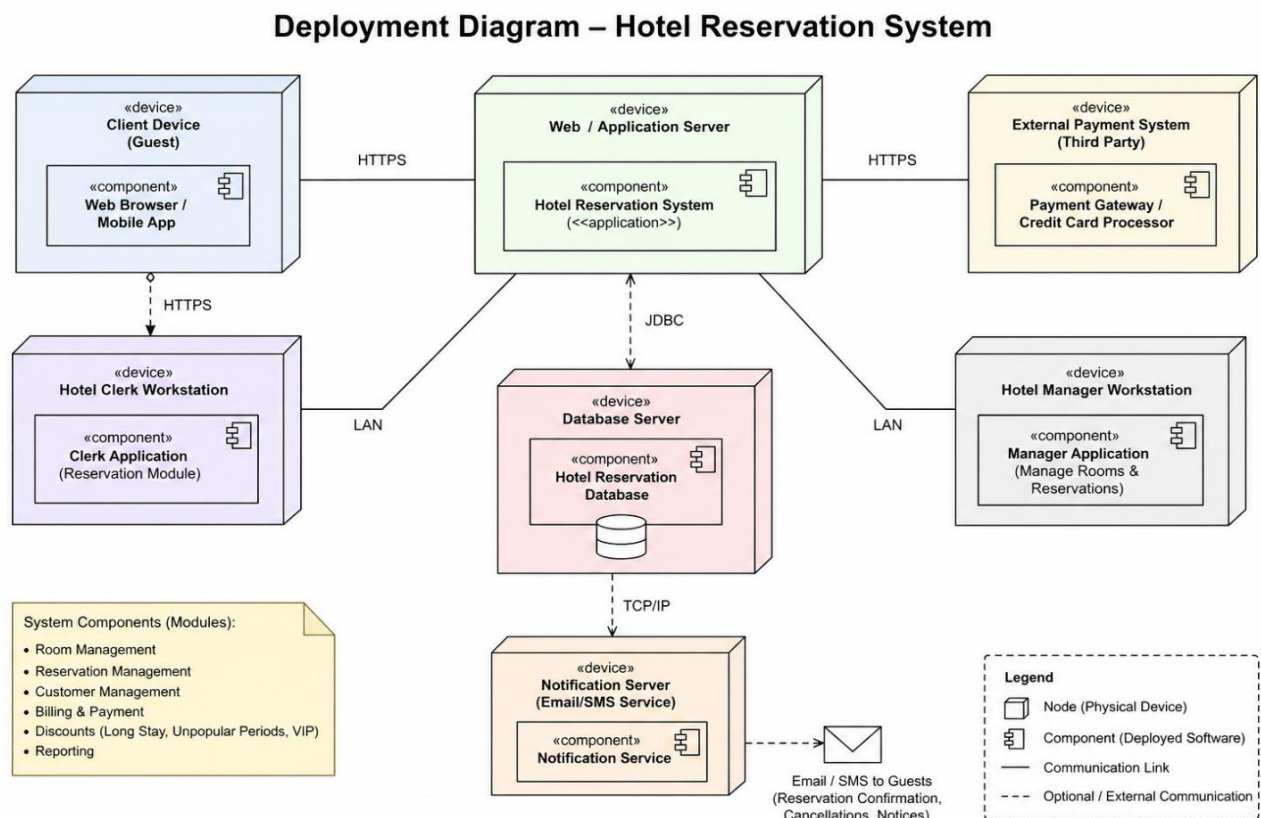
For VIP guests, the system allows to make unlimited number of reservations, these guests may include politicians, foreigners, tourists etc. In addition, several optional and variant capabilities are provided. For the hotel manager, system allows to alter reservation list i.e. remove rooms, add rooms, change room details (remember to issue notice to the future guests!)

In addition to that, system maintains list of rooms and reservations in the database, interact with external payment system for guest payments, apply discounts for long stays, unpopular periods of the year and for VIPs.

Task

Create deployment diagram for the provided scenario.

DIAGRAM:





Lab Report # 10

PACKAGE DIAGRAM

Objective

To learn how to create UML Package Diagrams by grouping related use cases and classes into logical packages using Microsoft Visio.

Introduction

A Package Diagram is a UML structural diagram used to organize related model elements into packages. It helps divide a software system into manageable modules, making the design easier to understand, maintain, and reuse. In this experiment, package diagrams are created for both the use cases and the classes of an Online Shopping Portal.

Tools Required

Microsoft Visio.
Windows Operating System.

Problem Statement

Design the UML model for an Online Shopping Portal where registered customers can browse products, manage shopping carts, place orders, and make payments. The administrator manages customers, products, categories, and inventory. Create the Use Case Diagram, Class Diagram, and organize both into appropriate packages.

Procedure

1. Study the given Online Shopping Portal scenario.
2. Identify all actors and use cases.
3. Draw the UML Use Case Diagram.
4. Group related use cases into packages.
5. Identify the system classes.
6. Draw the UML Class Diagram.
7. Group related classes into packages.
8. Create all diagrams using Microsoft Visio.



Diagram 1: Use Case Diagram

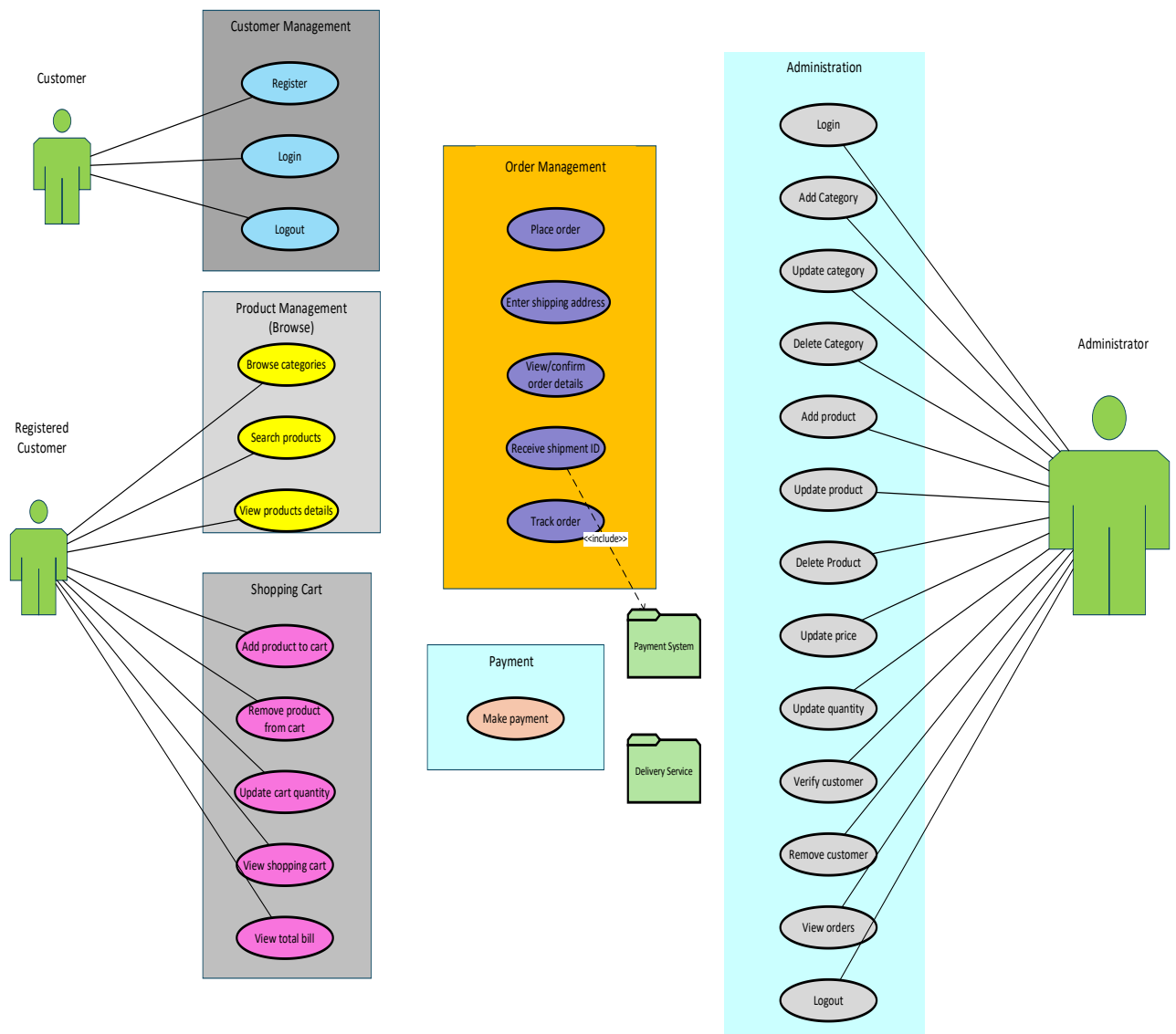




Diagram 2: Use Case Package Diagram

This package diagram groups related use cases into logical modules such as Customer Management, Shopping Cart, Product Browsing, Order Processing, and Administration.





Diagram 3: Class Diagram

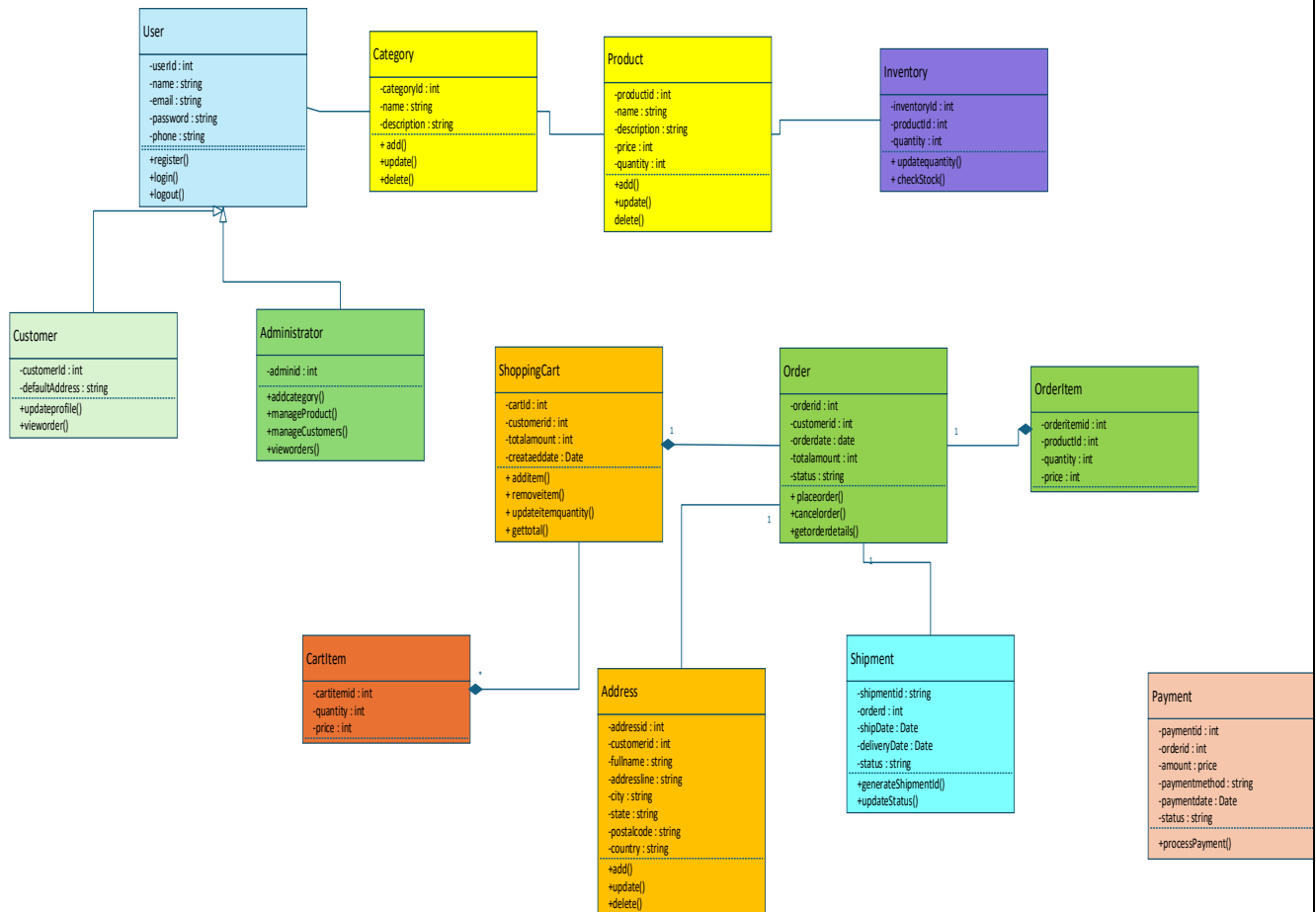
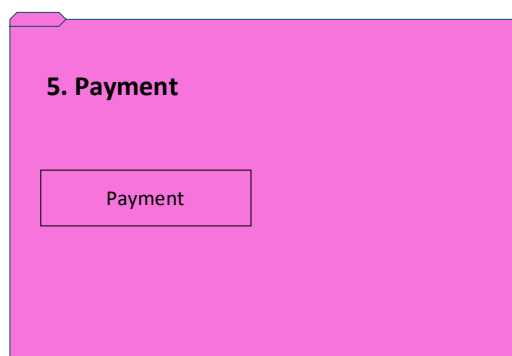
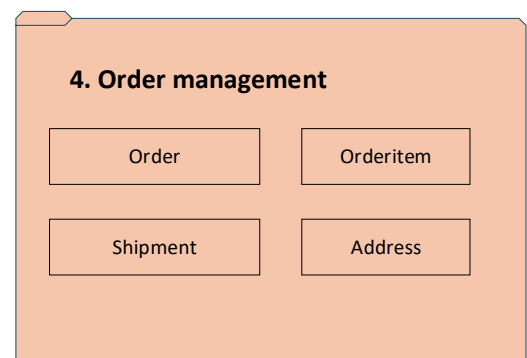
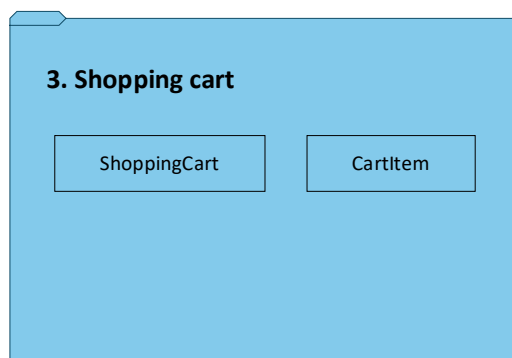
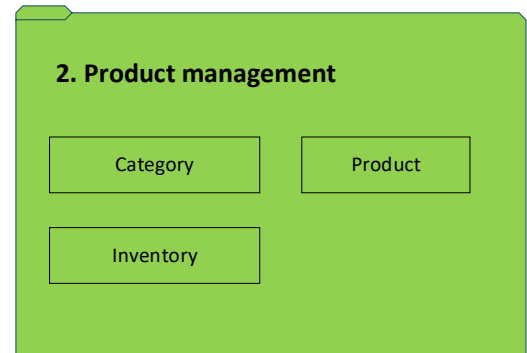
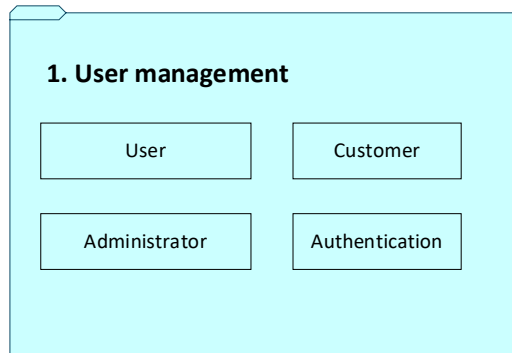




Diagram 4: Class Package Diagram

This package diagram groups related classes into logical packages such as Customer, Product Management, Shopping Cart, Order Processing, Payment, Shipment, and Administration.



Observation

Package diagrams improve software organization.

Related use cases can be grouped according to functionality.

Related classes can be grouped into modules for better maintainability.

Modular design makes large software systems easier to understand and extend.



Result

The Use Case Diagram, Use Case Package Diagram, Class Diagram, and Class Package Diagram for the Online Shopping Portal were successfully designed using Microsoft Visio. The system was organized into logical packages, demonstrating the concept of modular software design.

Conclusion

This experiment demonstrated how UML Package Diagrams are used to organize both use cases and classes into meaningful modules. By grouping related functionalities and classes together, the Online Shopping Portal becomes more structured, maintainable, reusable, and easier to understand. The experiment also provided practical experience in designing UML diagrams using Microsoft Visio.

Lab Report # 11

OPEN-ENDED LAB

Objective

The objective of this experiment is to analyse the given banking system requirements and design a UML-based solution using appropriate diagrams. The aim is to understand system functionality, identify system entities, and model their relationships and interactions using UML diagrams.

Problem Description

The Bank Accounts and Transactions System (BATS) is designed for Prime Bank Corporation to manage customer bank accounts and banking transactions.

The system provides the following services:

- Open bank account.
- Close bank account.
- Deposit money.
- Withdraw money.
- Transfer funds.
- Check account balance.
- Record all transactions.
- Generate monthly account statements.

Customers can access the system through:

- Teller.
- ATM.
- Internet Banking.

Different access methods support different banking services according to the given requirements. The system also manages different account types:



- Savings Account
- Checking Account

Savings accounts cannot be overdrawn, while checking accounts may allow overdrafts within an approved credit limit.

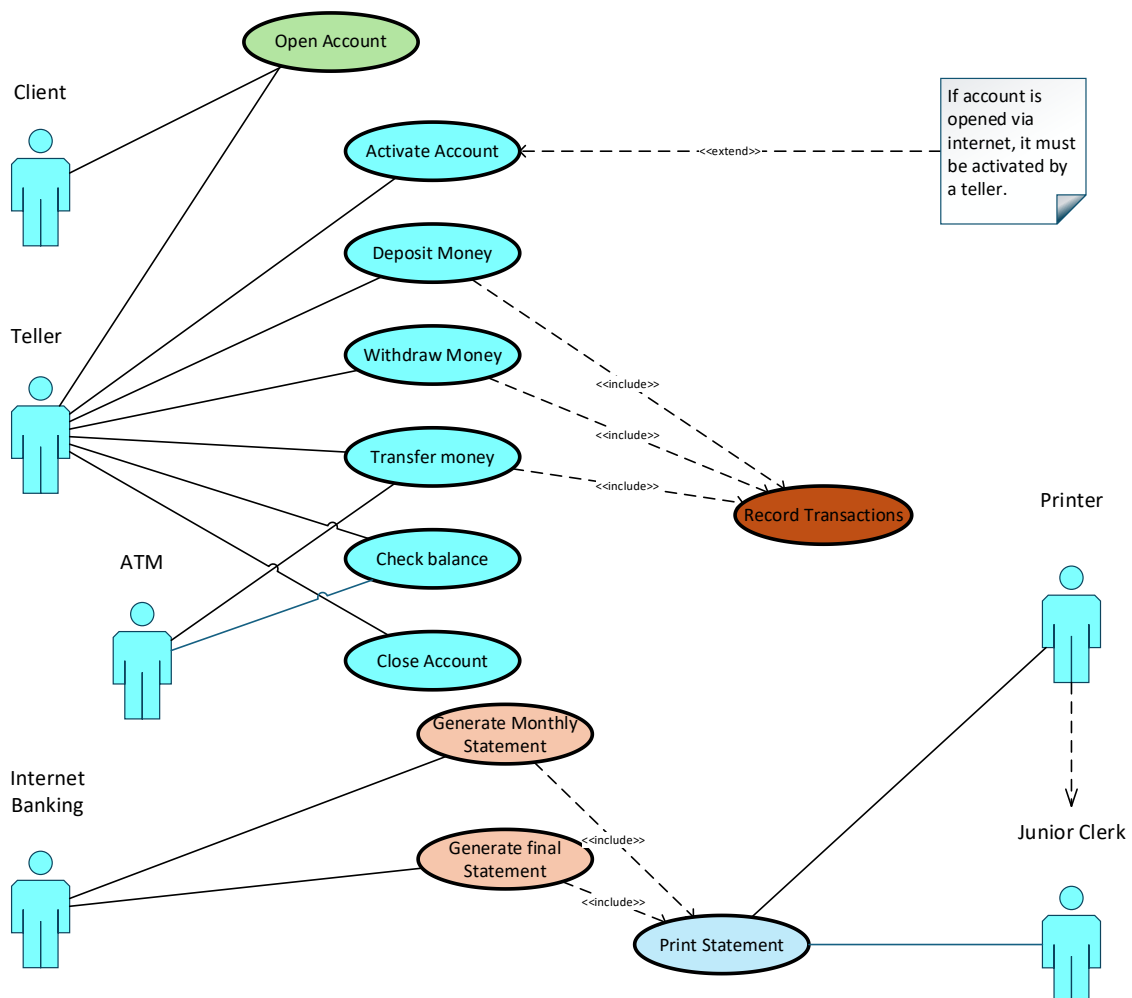
Software Used

- Microsoft Visio
- Microsoft Word

UML Diagrams

1. Use Case Diagram

The Use Case Diagram illustrates the interaction between different actors and the banking system. It identifies all services available through Teller, ATM, and Internet Banking.



Description of Use Case Diagram

The diagram contains the following actors:

- Client.
- Teller.



- ATM.
- Internet Banking.
- Printer.
- Junior Clerk.

Major use cases include:

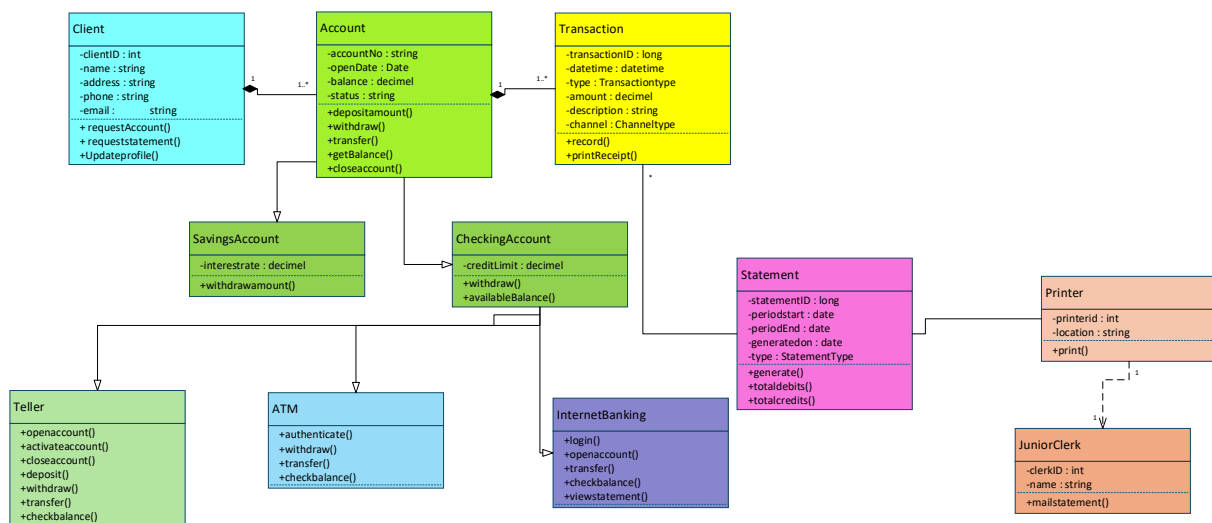
- Open Account.
- Activate Account.
- Close Account.
- Deposit Money.
- Withdraw Money.
- Transfer Funds.
- Check Balance.
- Generate Monthly Statement.
- Print Statement.

The diagram also represents restrictions such as:

- Deposits cannot be performed through ATM or Internet.
- Withdrawals cannot be performed through Internet Banking.
- Closing an account is only performed by the Teller.
- Internet-opened accounts require activation by the Teller.

2. Class Diagram

The Class Diagram represents the static structure of the Bank Accounts and Transactions System. It illustrates the classes, their attributes, operations, and relationships.





Description of Class Diagram

The class diagram contains major classes such as:

- Bank.
- Client.
- Account.
- Savings Account.
- Checking Account.
- Transaction.
- Statement.
- Teller.
- ATM.
- Internet Banking.
- Printer.

Relationships include:

- A Client can own one or more Accounts.
- An Account records multiple Transactions.
- Savings Account and Checking Account inherit from Account.
- Statements are generated from Transactions.
- Teller, ATM, and Internet Banking access banking services.

System Features

The designed system provides the following functionality:

- Account creation and activation.
- Account closure.
- Cash deposit.
- Cash withdrawal.
- Fund transfer.
- Balance inquiry.
- Transaction recording.
- Monthly statement generation.
- Statement printing.

Result

The Bank Accounts and Transactions System was successfully analyzed and modelled using UML diagrams. The Use Case Diagram represents the functional requirements and user interactions, while the Class Diagram represents the structural design of the system. These diagrams provide a clear understanding of the system architecture and can be used as the foundation for software implementation.



Conclusion

This experiment demonstrated the practical application of UML in designing a banking system. By developing the Use Case Diagram and Class Diagram, the system's functionality, actors, classes, and relationships were effectively modelled. UML provides a structured approach for software analysis and design, making complex systems easier to understand, communicate, and implement.

Lab Report # 12

MODEL VIEW CONTROLLER (MVC) IMPLEMENTATION

OBJECTIVE

The aim of this lab is to introduce students to the concept of Model View Controller Pattern. A small application shall be designed using this pattern.

INTRODUCTION

Model View Controller (MVC) is one of the most popular software architectural patterns. It separates an application into three different components.

- **Model** manages and stores application data.
- **View** provides the graphical interface through which users interact with the system.
- **Controller** receives user requests, processes them, communicates with the Model, and returns the appropriate View.

Because each component has a separate responsibility, the software becomes easier to understand, modify, and maintain.

SCENARIO

An Online Course Registration System is developed for university students.

At the beginning of every semester, students receive a course catalogue containing information about available courses, instructors, venues, schedules, and prerequisites.

Students register online by entering their personal information, previous academic record, scholarship information, degree program, and selecting the desired courses.

All information is stored in a database and managed through the MVC architecture

MVC Components

1. MODEL

The Model is responsible for storing and managing all application data.

It performs the following tasks:



- Stores student information
- Stores previous academic history
- Stores financial records
- Stores course information
- Stores registration details
- Retrieves information from the database whenever requested

DATABASE TABLES

Student Table

Attribute	Description
StudentID	Primary Key
Name	Student Name
RegistrationNo	Registration Number
ContactNumber	Contact Number
DegreeProgram	Degree Program

Academic History Table

Attribute	Description
AcademicID	Primary Key
StudentID	Foreign Key
SemesterGPA	Previous GPA
PreviousCourses	Previously Studied Courses

Financial Record Table

Attribute	Description
FinancialID	Primary Key
StudentID	Foreign Key
Scholarship	Scholarship Status



Course Table

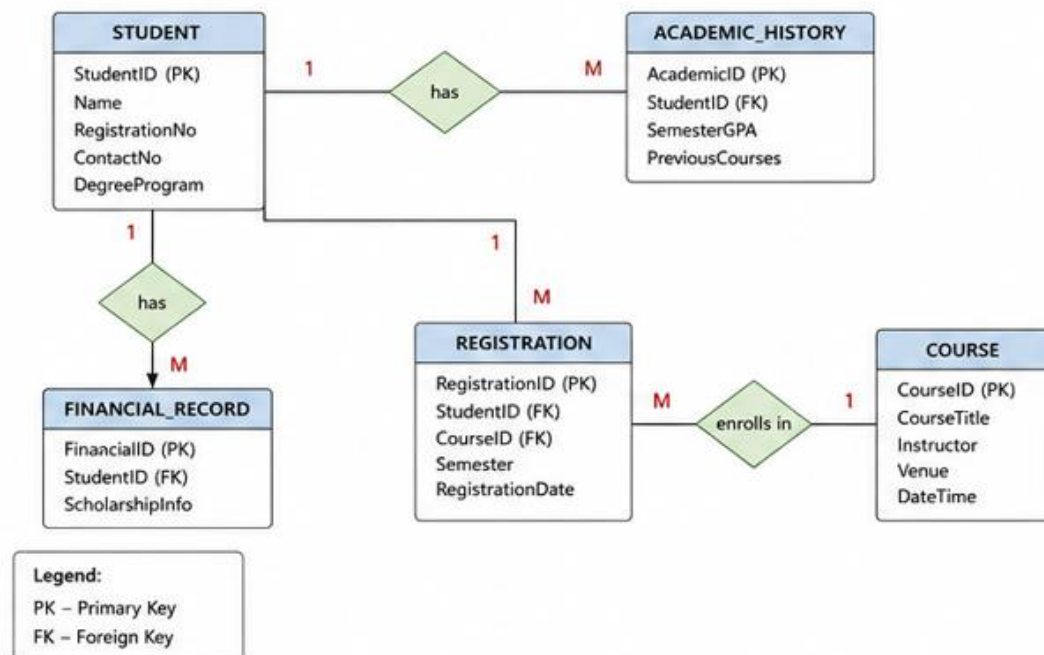
Attribute	Description
CourseID	Primary Key
CourseTitle	Subject Name
Instructor	Teacher Name
Venue	Classroom
DateTime	Schedule

Registration Table

Attribute	Description
RegistrationID	Primary Key
StudentID	Foreign Key
CourseID	Foreign Key
Semester	Current Semester

ER Diagram

1. ER DIAGRAM (MODEL / DATABASE DESIGN)



2. View

The View is responsible for displaying information to the user.



It allows students to interact with the registration system through different screens.

The View includes:

- Login Screen
- Student Registration Form
- Course Selection Screen
- Registration Success Screen

Login Screen

The student enters:

- Username
- Password

After successful login, the registration form is displayed.

Student Registration Form

The form contains:

- Name
- Registration Number
- Contact Number
- Degree Program
- Semester GPA
- Scholarship Information

Course Selection Screen

The student can:

- View available courses
- Select desired courses
- Submit registration

Registration Success Screen

After successful registration, the system displays:

- Student Name
- Registration Number
- Semester
- Registered Courses

INTERFACE DESIGN



VIEW (INTERFACE DESIGN / WIREFRAMES)

1. LOGIN SCREEN

**ONLINE COURSE
REGISTRATION SYSTEM**



Username

Password

LOGIN

New Student? [Register Here](#)

2. STUDENT INFORMATION FORM

Student Information

Name

Registration No.

Contact Number

Degree Program

Semester GPA

Scholarship ☐ Yes ☐ No

NEXT

3. COURSE REGISTRATION SCREEN

Available Courses

Select	Course Title	Instructor	Time
☐	Database Systems	Dr. Ali	Mon 10:00 AM
☐	Artificial Intelligence	Dr. Sara	Tue 11:00 AM
☐	Computer Networks	Dr. Khan	Wed 09:00 AM
☐	Software Design	Dr. Ahmed	Thu 02:00 PM
☐	Operating Systems	Dr. Usman	Fri 10:00 AM

Selected Courses

1. Database Systems
2. Computer Networks

SUBMIT REGISTRATION

4. REGISTRATION SUCCESS SCREEN



Registration Successful!

Name : Ali Ahmed
Reg. No. : 22-SE-101
Semester : Fall 2024

Registered Courses

✓ Database Systems
✓ Computer Networks

BACK TO HOME

3. Controller

The Controller acts as the bridge between the Model and the View.

It receives requests from the user, validates the data, processes the request, communicates with the database, and displays the final result.

CONTROLLER RESPONSIBILITIES

Login Controller

- Verifies username and password.
- Grants access if the credentials are correct.

Student Controller

- Saves student information.
- Updates student profile.
- Validates entered information.

Course Controller

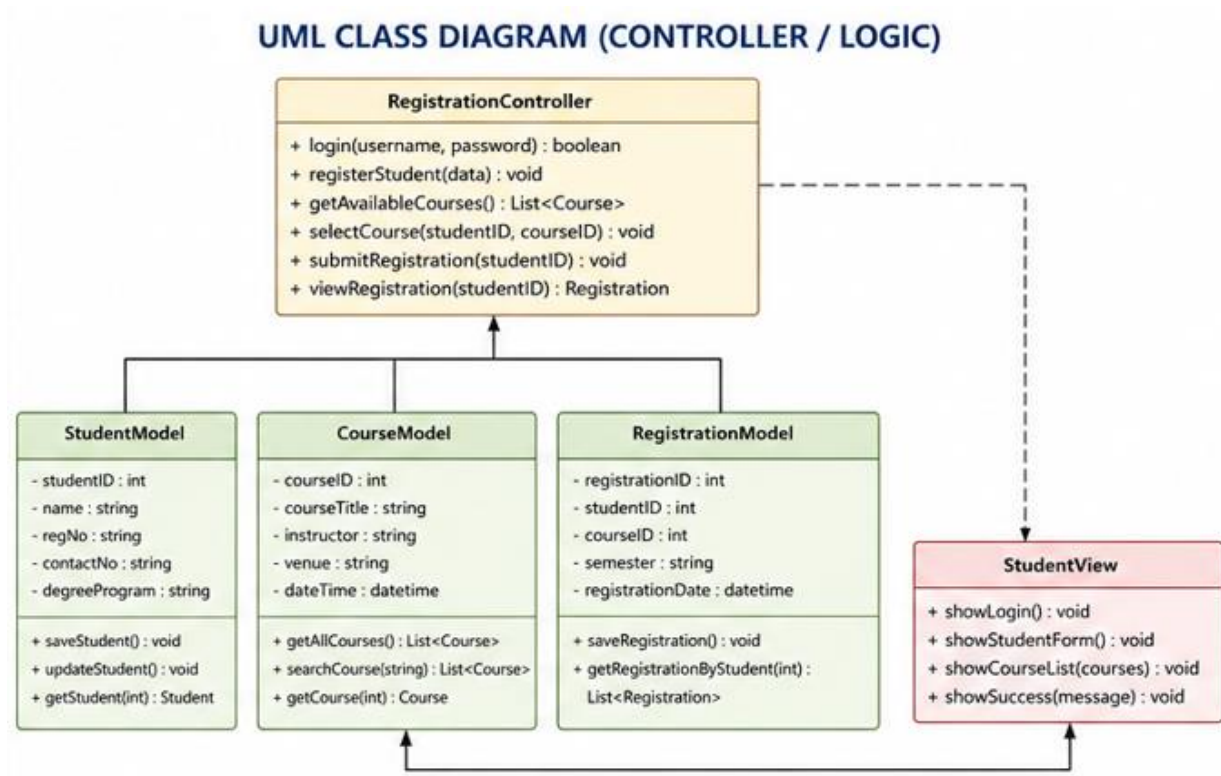
- Displays available courses.
- Allows students to select courses.
- Checks prerequisites before registration.

Registration Controller

- Stores registration details.
- Saves selected courses.
- Displays registration confirmation.



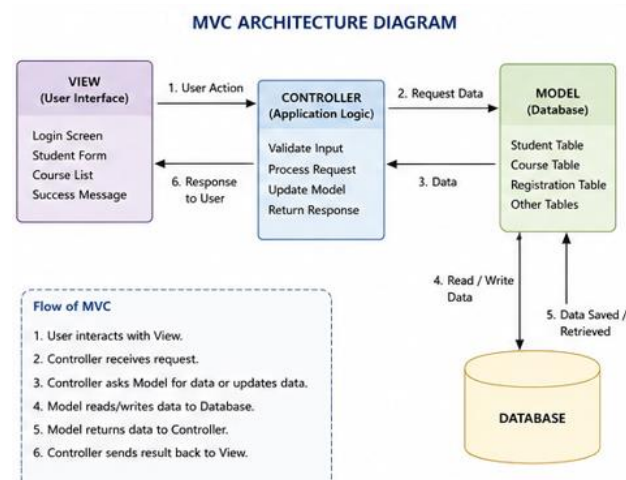
UML Class Diagram



MVC Architecture Diagram

The MVC architecture works in the following order:

1. The user interacts with the View.
2. The View sends the request to the Controller.
3. The Controller validates the request.
4. The Controller requests data from the Model.
5. The Model reads or writes data in the database.
6. The Model returns the result to the Controller.
7. The Controller updates the View.
8. The View displays the final result to the user.





Working of the System

1. The student opens the Online Registration System.
2. The Login Screen is displayed.
3. The student enters username and password.
4. After login, the student fills the registration form.
5. Personal and academic information is entered.
6. The student selects desired courses.
7. The Controller validates the entered information.
8. The Model stores the information in the database.
9. The registration is successfully completed.
10. A confirmation message is displayed.

Advantages of MVC

- Separates data from user interface.
- Easy to understand.
- Easy to maintain.
- Supports code reusability.
- Simplifies testing.
- Multiple developers can work independently.
- Makes the application scalable

CONCLUSION

In this lab, an **Online Course Registration System** was designed using the **Model View Controller (MVC)** architectural pattern. The Model manages the database, the View provides the user interface, and the Controller handles all application logic and communication between the Model and View. MVC improves software organization, maintainability, scalability, and overall development efficiency.
